Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-02
Data Preprocessing and Linear Regression

1.	Write a Python program using Pandas to load a student performance dataset and display its first five records, shape, column names, missing values, and statistical summary.
	Solution: import **pandas** as **pd** # *Load dataset* data = **pd**.**read_csv**("data/student_performance.csv") **print**("FIRST FIVE RECORDS") **print**(data.**head**()) **print**("\nDATASET SHAPE") **print**(data.shape) **print**("\nCOLUMN NAMES") **print**(data.columns) **print**("\nMISSING VALUES") **print**(data.**isnull**().**sum**()) **print**("\nSTATISTICAL SUMMARY") **print**(data.**describe**())
2.	Write a Python program using TensorFlow to predict students' final marks from attendance, internal marks, assignment marks, study hours, and previous marks using a neural network regression model.
	Solution: import **pandas** as **pd** import **numpy** as **np** import **tensorflow** as **tf** import **joblib** from **sklearn**.**model_selection** import **train_test_split** from **sklearn**.**preprocessing** import **StandardScaler** data = **pd**.**read_csv**("data/student_performance.csv") X = data[["Attendance",

```
        "Internal_Marks",
        "Assignment_Marks",
        "Study_Hours",
        "Previous_Marks"
    ]
]

y = data["Final_Marks"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# SCALING
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Save scaler
joblib.dump(
    scaler,
    "scalers/marks_scaler.pkl"
)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(5,)),
    tf.keras.layers.Dense(1)
])
model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    validation_split=0.2,
    verbose=1
)
loss, mae = model.evaluate(
    X_test,
    y_test,
    verbose=0
)
print("\nTest MAE:", mae)
```

	model.save("models/marks_model.keras") **print**("\nMarks model saved successfully."

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-03
Multilayer Perceptron (MLP)

1.	Write a Python program using TensorFlow to classify students as “At Risk” or “Not At Risk” based on attendance, internal marks, assignment marks, study hours, and previous marks using a Multilayer Perceptron (MLP) model.

Solution:

```
import pandas as pd
import tensorflow as tf
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# ==========================================
# LOAD DATA
# ==========================================

data = pd.read_csv(
    "data/student_performance.csv"
)
# ==========================================
# CREATE RISK LABEL
# ==========================================

data["Risk"] = (
    data["Final_Marks"] < 60
).astype(int)

# ==========================================
# INPUT
# ==========================================

X = data[
    [
        "Attendance",
        "Internal_Marks",
        "Assignment_Marks",
        "Study_Hours",
        "Previous_Marks"
    ]
]

# ==========================================
```

```
# TARGET
# ============================================

y = data["Risk"]

# ============================================
# SPLIT
# ============================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# ============================================
# SCALE
# ============================================

scaler = StandardScaler()

X_train = scaler.fit_transform(
    X_train
)

X_test = scaler.transform(
    X_test
)

joblib.dump(
    scaler,
    "scalers/risk_scaler.pkl"
)

# ============================================
# MLP MODEL
# ============================================

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(5,)),
    tf.keras.layers.Dense(16,activation="relu"),
    tf.keras.layers.Dense(8,activation="relu"),
    tf.keras.layers.Dense(1,activation="sigmoid")
])
```

```
# ================================================
# COMPILE
# ================================================

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# ================================================
# TRAIN
# ================================================

model.fit(
    X_train,
    y_train,
    epochs=50,
    validation_split=0.2,
    verbose=1
)

# ================================================
# EVALUATE
# ================================================

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print(
    "\nTest Accuracy:",
    accuracy
)

# ================================================
# SAVE
# ================================================

model.save(
    "models/risk_model.keras"
)

print(
    "Risk model saved successfully."
```

	)

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-04
Convolutional Neural Networks (Classification)

1.	Write a Python program using TensorFlow to classify handwritten digits from the MNIST dataset using a Convolutional Neural Network (CNN), evaluate its accuracy, and save the trained model.
	Solution:

```
import tensorflow as tf
# Load MNIST
(X_train, y_train), (X_test, y_test) = \
    tf.keras.datasets.mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# Reshape
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
# CNN Model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(
        32, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation="relu"
    ),
    tf.keras.layers.Dense(10,
        activation="softmax")
])

# Compile
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
# Train
model.fit(
    X_train,
    y_train,
    epochs=3,
    batch_size=64
)
```

	# Test loss, accuracy = model.evaluate(X_test, y_test, verbose=0) **print**("CNN Accuracy:", accuracy) *# Save model* model.save("cnn_model.keras") **print**("Model saved successfully!")

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-05

Autoencoders

1.	Write a Python program using TensorFlow to build and train an Autoencoder on the MNIST dataset for image reconstruction, and save the trained model.

Solution:

```
import tensorflow as tf
(X_train, _), (X_test,_) = tf.keras.datasets.mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)
input_layer = tf.keras.Input(shape=(784,))
# Encoder
encoded = tf.keras.layers.Dense(64, activation="relu"
)(input_layer)

encoded = tf.keras.layers.Dense(
    16,
    activation="relu"
)(encoded)

# Decoder
decoded = tf.keras.layers.Dense(
    64,
    activation="relu"
)(encoded)

decoded = tf.keras.layers.Dense(
    784,
    activation="sigmoid"
)(decoded)

# Complete model
autoencoder = tf.keras.Model(
    input_layer,
    decoded
)

autoencoder.compile(
    optimizer="adam",
    loss="binary_crossentropy"
)
autoencoder.fit(
    X_train,
```

```
    X_train,
    epochs=5,
    batch_size=256,
    validation_data=(
        X_test,
        X_test
    )
)
autoencoder.save(
    "models/autoencoder.keras"
)

print(
    "Autoencoder saved."
)
```

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-06

Recurrent Neural Network (RNN)

1.	Write a Python program using TensorFlow to perform sentiment classification of IMDB movie reviews using an Embedding layer and SimpleRNN, and save the trained model.
	Solution: import **tensorflow** as **tf** from **tensorflow**.keras.datasets import imdb from **tensorflow**.keras.preprocessing.sequence import pad_sequences (X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000) X_train = pad_sequences(X_train, maxlen=200) X_test = pad_sequences(X_test, maxlen=200) model = **tf**.**keras**.Sequential([**tf**.**keras**.**layers**.**Embedding**(10000, 32), **tf**.**keras**.**layers**.SimpleRNN(32), **tf**.**keras**.**layers**.**Dense**(1, activation="sigmoid")]) model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"]) model.fit(X_train, y_train, epochs=3, batch_size=128, validation_data=(X_test, y_test)) model.save("sentiment_rnn.keras") **print**("Model saved successfully!")

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-07

Long Short Term Memory Network (LSTM)

1.	Write a Python program using TensorFlow to predict the next value in a sequence using an LSTM neural network, train the model on sequential data, and save the trained model.

Solution:

```
import numpy as np
import tensorflow as tf
X = []
y = []
for i in range(1, 101):
    X.append([
        i,
        i + 1,
        i + 2
    ])

    y.append(
        i + 3
    )

X = np.array(X)
y = np.array(y)

# ================================================
# RESHAPE
# ================================================

X = X.reshape(
    X.shape[0],
    X.shape[1],
    1
)

# ================================================
# LSTM MODEL
# ================================================

model = tf.keras.Sequential([

    tf.keras.layers.Input(
        shape=(3, 1)
    ),

    tf.keras.layers.LSTM(
```

```
        32
    ),

    tf.keras.layers.Dense(
        1
    )
])

# ================================================
# COMPILE
# ================================================

model.compile(
    optimizer="adam",
    loss="mse"
)

# ================================================
# TRAIN
# ================================================

model.fit(
    X,
    y,
    epochs=100,
    verbose=0
)

# ================================================
# SAVE
# ================================================

model.save(
    "models/lstm_model.keras"
)

print(
    "LSTM model saved."
)
```

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-08

Regularization Techniques in Deep Learning

1.	Write a Python program using TensorFlow to classify Fashion-MNIST images using a neural network with Dropout regularization and Early Stopping, and plot the training and validation loss to observe overfitting.

Solution:

```
import tensorflow as tf
import matplotlib.pyplot as plt
(
    X_train,
    y_train
), (
    X_test,
    y_test
) = tf.keras.datasets.fashion_mnist.load_data()
X_train = X_train[:10000]
y_train = y_train[:10000]
X_test = X_test[:2000]
y_test = y_test[:2000]
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
model = tf.keras.Sequential([
    tf.keras.layers.Input(
        shape=(28, 28)
    ),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(
        128,
        activation="relu"
    ),

    tf.keras.layers.Dropout(
        0.3
    ),

    tf.keras.layers.Dense(
        64,
        activation="relu"
    ),

    tf.keras.layers.Dropout(
        0.3
    ),
```

```
    tf.keras.layers.Dense(
        10,
        activation="softmax"
    )
])

# ================================================
# COMPILE
# ================================================

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    validation_split=0.2,
    callbacks=[early_stopping]
)

# ================================================
# GRAPH
# ================================================

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel(
    "Epoch"
)
```

	plt.**ylabel**("Loss") **plt**.**title**("Regularization and Overfitting") **plt**.**legend**() **plt**.**show**()

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-09

Deep feed forward Network

1.	Write a Python program to implement a Deep Feed-Forward Network with two hidden layers for multiclass classification and predict the class of a sample input.

	Solution:
	import tensorflow as tf
	import numpy as np
	
	# Load MNIST dataset
	(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
	
	# Normalize and flatten
	X_train = X_train.reshape(-1, 784) / 255.0
	X_test = X_test.reshape(-1, 784) / 255.0
	
	# Deep Feed Forward Network
	model = tf.keras.Sequential([
	  tf.keras.layers.Input(shape=(784,)),
	  tf.keras.layers.Dense(128, activation="relu"),
	  tf.keras.layers.Dense(64, activation="relu"),
	  tf.keras.layers.Dense(10, activation="softmax")
	])
	
	# Compile
	model.compile(
	  optimizer="adam",
	  loss="sparse_categorical_crossentropy",
	  metrics=["accuracy"]
	)
	
	# Train
	model.fit(X_train, y_train, epochs=5, batch_size=64)
	
	# Evaluate
	loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
	print("Test Accuracy:", accuracy)
	
	# Predict class
	prediction = model.predict(X_test[:1])
	print("Predicted Class:", np.argmax(prediction))
	print("Actual Class:", y_test[0])

Smt. Chandibai Himathmal Mansukhani College (Autonomous)

T.Y.B.Sc. Data Science SEM-V [2026-2027]

Deep Learning

Practical-10

Variational Autoencoder (VAE)

1.	Write a Python program to implement a Variational Autoencoder (VAE) for learning latent representations and generating new data samples using MNIST.

Solution:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 784) / 255.0

# Encoder
encoder = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(2)
])

# Decoder
decoder = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(784, activation="sigmoid")
])

# VAE model
class VAE(tf.keras.Model):
    def __init__(self):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def call(self, x):
        z = self.encoder(x)
        return self.decoder(z)

vae = VAE()

vae.compile(
    optimizer="adam",
    loss="mse"
)
```

```
# Train
vae.fit(
    X_train,
    X_train,
    epochs=5,
    batch_size=128
)

# Generate new samples
z = np.random.normal(size=(5, 2))
images = decoder.predict(z)

# Display
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(images[i].reshape(28, 28), cmap="gray")
    plt.axis("off")

plt.show()

print("VAE trained successfully!")
```